

餘燼航路：全域開發報告

《小白龍核心母站 v3.2.1：網站製作》專案總結報告

報告日期： 2026-05-12
數據來源： 總部 (acer) 自動入庫分析

I. 整體開發進度

本專案的核心成果是《小白龍核心母站 v3.2.1》Streamlit 應用程式的建構與穩定。截至目前為止，該系統已展現高度成熟與功能豐富的特性，旨在作為一個資深的遊戲開發助手，特別針對《餘燼航路》專案提供全方位的支持。

核心系統狀態與主要進展：

1. **高沉浸主題與介面設計：**
* 應用程式名稱與所有介面文字、提示、功能描述皆與「小白龍」及「餘燼航路」的末世機甲主題緊密結合，創造出高度沉浸且吸引人的使用者體驗。
2. **先進的多模態 AI 整合：**
* 成功利用 Google Gemini 的多模態能力，實現對語音、圖像、音訊和文字的混合輸入解析，極大地提升了開發者捕捉靈感和進行跨模態分析的效率。
3. **完善的檔案與專案管理機制：**
* 建立了清晰的日誌 (logs) 和媒體檔案 (media) 分專案儲存結構，便於數據的歸檔與檢索。
* 實施了 initialize_environment() 自癒機制，確保必要的物理空間 (資料夾) 在每次執行時自動創建或重構。
* 支援多專案切換、新增、修改與刪除，使得工具具備高度的彈性與可擴展性。
4. **強大的 PDF 知識庫與報告生成：**
* 引入 PyPDF2 支援 PDF 文件導入，作為 AI 參考的知識來源，極大地擴展了 AI 的上下文理解能力。
* 整合 weasyprint 導出功能，能夠生成格式精美的開發優化報告、修復報告、全域諮詢建議及專案總結報告 PDF。
5. **自動化雲端同步 (Git)：**
* secure_auto_push 函數實現了一鍵自動同步至 GitHub 的功能，簡化了版本控制流程，提升了團隊協作與數據安全。

6. **獨特的「航道啟示錄」頻道：**

* 結合隨機媒體素材（圖片、音效）與富含哲學語調的 AI 指引，為開發者在迷霧中提供靈感啟示，增添了工具的趣味性與實用性。

7. **優化的 API Key 管理：**

* 透過 `st.session\_state` 有效管理 API Key，並支援從記憶清單或手動輸入，提升了安全性和便捷性。

總體而言，此母站系統已具備作為核心開發輔助工具的潛力，其創新設計與實用功能將對《餘燼航路》的開發進程產生積極影響。

II. 各城市發射站的靈感彙整

根據本次自動入庫分析的數據，主要的靈感與系統架構源自「總部 (acer)」發射站。儘管系統設計上已規劃多個發射站點，但當前紀錄集中反映總部的開發活動。

1. 總部 (acer) - 核心主機 (2026-05-12 20:17:43)

* **靈感核心：** 將整個開發工具打造成一個「核心母站」，其介面與功能皆圍繞「小白龍」和「餘燼航路」的末世、黃銅蒸氣與機甲科幻主題展開。這種高度風格化的設計旨在為開發者提供一個沉浸式的「作戰指揮中心」，而非僅僅是一個冰冷的程式介面。

* **具體展現：**

* **「素材打撈 (Media)」頻道：

類比雷達站，用於捕捉來自各地的開發素材與靈感。

* **「齒輪重組 (Script)」與「結構修復 (Patch)」頻道：

比喻為對核心機甲邏輯與部件進行精密維護與升級。

* **「虛空啟示 (Oracle)」與「航道啟示錄 (Oracle's Compass)」頻道：

** 透過 AI 扮演的「架構導師」或「古老導靈」，提供哲學性、策略性的指引與預言，幫助開發者在迷霧中找到方向。

* **「航行日誌 (Log)」：

記錄所有開發活動與 AI 分析報告的「歷史航跡」。

* **關聯音訊：

無。

* **指令：

完整載入並初始化 Streamlit 應用程式的環境與核心功能，即本次提交的代碼本身。這表明總部主要負責核心架構的設計與部署。

2. 其他發射站 (台北站、新竹站、台南站)

* 根據系統的 UI 設計，已預留台北站、新竹站、台南站作為遠端開發節點。系統具備從這些站點進行「素材打撈」及「全專案大數據彙整」的能力，旨在整合來自不同地理位置的開發資源與靈感。

* **當前狀態：

本次提交的日誌數據中，尚未有來自這些遠端站點的具體開發素材或靈感紀錄。這些站點的潛力將在未來的開發迭代中逐步實現，以豐富專案的多樣性與廣度。未來可期待從這些地點獲取更多元且具地域特色的開發洞察。

III. 待辦清單 (To-Do List)

基於 AI 綜合分析報告，以下列出系統當前待處理的事項，區分為不同優先級：

A. 關鍵錯誤與潛在問題 (Critical/High Priority)

1. **修正 `initialize\_environment()` 中的 `PROJECT\_ROOT` 未定義錯誤：**

* **問題描述：** 程式碼頂部定義了 `DATA\_ROOT`，但在 `initialize\_environment()` 函數中誤用了未定義的 `PROJECT\_ROOT` 變數。

* **解決方案：** 將 `initialize\_environment()` 中 `folders\_to\_create` 列表裡的 `PROJECT\_ROOT` 改為 `DATA\_ROOT`。

2. **重新評估 `secure\_auto\_push` 中的 `git push --force` 用法：**

* **問題描述：** 強制推送 (`--force`) 可能導致遠端分支歷史被覆蓋，存在數據丟失風險，尤其在協作環境下極為危險。

* **解決方案：** 移除 `--force`。或考慮提供選項讓使用者明確選擇是否強制推送並給予顯著警告。更佳實踐是在推送前嘗試 `git pull origin main --rebase` 處理潛在衝突。

3. **優化 `LOG\_DIR` 和 `MEDIA\_DIR` 的定義與創建邏輯：**

* **問題描述：** 程式碼頂部和專案切換後重複定義 `LOG\_DIR` 和 `MEDIA\_DIR`，可能導致在 `all\_projects` 根目錄下創建不必要的空 `logs` 和 `media` 資料夾。

* **解決方案：** 移除程式碼頂部的全局定義，並確保僅在選定 `current\_p\_name` 專案後，才創建該專案特定的子目錄 (`all\_projects/logs` 和 `all\_projects/media`)。

4. **改進 `genai.configure` 的呼叫效率：**

* **問題描述：** `genai.configure(api\_key=FINAL\_KEY)` 位於 Streamlit 腳本主體中，每次重新運行時都會執行，效率不高。

* **解決方案：** 確保 `FINAL\_KEY` 確定並非空值後，只呼叫一次 `genai.configure`，或在需要時進行檢查避免重複配置。

B. 功能性改進與優化 (Medium Priority)

1. **大型專案的 AI Context 處理機制：**

* **問題描述：** 「虛空啟示」與「航行日誌」頻道會讀取所有 `.md` 和 `.gd` 檔案內容給 AI，大型專案可能面臨記憶體溢出、API token 限制及 AI 處理效率下降的問題。

* **解決方案：** 考慮實施內容摘要、基於查詢的檢索、分批處理等策略，或在接近 token 限制時給予使用者警告。

2. **動態`.gitignore`管理：**

問題描述：`.gitignore` 內容目前是硬編碼的，專案有特殊忽略需求時需手動修改。

解決方案：在「核心維護 (System)」頻道中提供一個文本區域，允許使用者編輯並儲存`.gitignore` 內容。

3. **Git 使用者資訊配置：**

問題描述：`git config --global user.email` 和 `user.name` 在 `secure\_auto\_push` 中被硬編碼。

解決方案：將這些資訊從`.env` 讀取，或在「核心維護」頻道中提供輸入框讓使用者自行設定。

4. **`.weasyprint` 依賴提醒：**

問題描述：`.weasyprint` 在某些系統上需要額外的系統級依賴。

解決方案：在應用程式的「核心維護」頻道或 README 中加入相關的安裝說明。

5. **提升錯誤訊息的詳細程度：**

問題描述：許多 `except Exception as e:` 捕獲所有錯誤，導致使用者看到的錯誤訊息可能不夠具體。

解決方案：針對常見的錯誤類型（如 `FileNotFoundError`、網路請求失敗、API 錯誤）進行更精確的捕獲與處理，提供更友善且具體的提示。

C. 小細節與程式碼風格 (Low Priority)

1. **PDF 內容導出的 HTML 轉義：**

問題描述：在導出 PDF 時，AI 回應的文本內容若包含特殊 HTML 字符，可能無法正確渲染。

解決方案：使用 `html.escape()` 函數更安全地處理 AI 回應的文本，確保所有特殊字符正確轉義。

2. **專案移除的確認機制：**

問題描述：刪除專案是高風險操作，目前只有一個按鈕。

解決方案：增加一個確認對話框或要求使用者再次輸入專案名稱以進行確認，防止誤觸。

已入庫語音清單

(本次數據未包含具體語音檔案列表，若有，應列於此處。)